# PLONK와 halo2의 관계

## PLONK 프로토콜 개요

**PLONK**(Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge)는 2019년에 발표된 범용 영지식 증명 시스템입니다. halo2는 PLONK 프로토콜을 기반으로 하되, 여러 개선사항을 추가한 구현체입니다.

### PLONK의 핵심 아이디어

1. **Universal Setup**: 모든 회로에 사용할 수 있는 범용 설정
2. **Polynomial Commitment**: 다항식 약속을 통한 효율적인 증명
3. **Permutation Argument**: 순열 논증을 통한 복사 제약조건 처리
4. **Lookup Argument**: 테이블 검색을 통한 복잡한 연산 처리

## PLONK 프로토콜의 기본 구조

### 1. 산술화 (Arithmetization)

PLONK는 계산을 **산술 회로**로 표현합니다:

```
회로의 제약조건:

q_L(X) · a(X) + q_R(X) · b(X) + q_O(X) · c(X) + q_M(X) · a(X) · b(X) + q_C(X) =
```

여기서:

- `a(X)`, `b(X)`, `c(X)`: 좌측, 우측, 출력 값들의 다항식

- `q_L`, `q_R`, `q_O`, `q_M`, `q_C` : 선택자(selector) 다항식들

### 2. 다항식 표현

모든 값들을 **라그랑주 기저**를 사용한 다항식으로 표현:

```
f(X) = Σ f_i · L_i(X)
```

여기서 `L_i(X)` 는 i번째 라그랑주 기저 다항식입니다.

### 3. 제약조건 검증

증명자는 모든 제약조건이 만족됨을 다음과 같이 증명:

```
gate_constraint(X) = q_L(X)·a(X) + q_R(X)·b(X) + q_O(X)·c(X) + q_M(X)·a(X)·b(X)
```

이 다항식이 모든 점에서 0이 됨을 증명합니다.

## halo2의 PLONK 구현

### 1. 기본 PLONK 구조 유지

halo2는 PLONK의 핵심 구조를 그대로 유지합니다:

```
// halo2에서의 Gate 정의 (PLONK의 제약조건과 동일)
let gate = meta.create_gate("basic_gate", |meta| {
let a = meta.query_advice(advice[0], Rotation::cur());
let b = meta.query_advice(advice[1], Rotation::cur());
let c = meta.query_advice(advice[2], Rotation::cur());
```

```
let q_l = meta.query_fixed(q_left, Rotation::cur());
let q_r = meta.query_fixed(q_right, Rotation::cur());
let q_o = meta.query_fixed(q_output, Rotation::cur());
let q_m = meta.query_fixed(q_mul, Rotation::cur());
let q_c = meta.query_fixed(q_const, Rotation::cur());
vec![q_l * a + q_r * b + q_o * c + q_m * a * b + q_c]
});
```

## 2. 확장된 Column 시스템

halo2는 PLONK의 기본 3개 컬럼(a, b, c)을 확장:

```
// PLONK: 고정된 3개 컬럼
// a, b, c

// halo2: 유연한 다중 컬럼
let advice_columns: [Column<Advice>; N] = [
meta.advice_column(),
meta.advice_column(),
// ... 필요한 만큼 추가 가능
];
```

## 3. 향상된 Lookup 시스템

### 기본 PLONK Lookup

```
// 단순한 테이블 검색
(a, b) ∈ Table
```

**halo2 Lookup**

```
// 복잡한 표현식과 다중 컬럼 지원
meta.lookup("complex_lookup", |meta| {
let  a = meta.query_advice(advice[0], Rotation::cur());
let  b = meta.query_advice(advice[1], Rotation::cur());
let  table_a = meta.query_fixed(table[0], Rotation::cur());
let  table_b = meta.query_fixed(table[1], Rotation::cur());
vec![(a + b, table_a + table_b)]
});
```

## halo2의 주요 개선사항

### 1. Recursive Proof 지원

**기본 PLONK**: 재귀적 증명 지원 제한적

**halo2**: 네이티브 재귀적 증명 지원

```
// halo2에서 재귀적 증명 검증
impl Circuit<F> for RecursiveCircuit {
fn  synthesize(&self, config: Self::Config, layouter: impl Layouter<F>) -> Resul
```

```
// 이전 증명을 검증하는 회로 구현
let previous_proof = self.previous_proof;
// 검증 로직...
}
}
```

## 2. 유연한 Constraint System

**기본 PLONK**: 고정된 Gate 구조

**halo2**: 사용자 정의 Gate 지원

```
// 복잡한 사용자 정의 Gate
let custom_gate = meta.create_gate("custom", |meta| {
let inputs: Vec<_> = (0..8).map(|i| {
meta.query_advice(advice[i], Rotation::cur())
}).collect();
// 복잡한 제약조건 정의
vec![
inputs[0] * inputs[1] - inputs[2],
inputs[3].square() - inputs[4],
// ... 더 많은 제약조건
]
});
```

### 3. 효율적인 Memory Layout

**기본 PLONK**: 단순한 행렬 구조

**halo2**: Region 기반 메모리 관리

```
// Region을 통한 효율적인 메모리 사용
layouter.assign_region(
|| "computation_region",
|mut region| {
// 지역적 메모리 할당
let a = region.assign_advice(|| "a", advice[0], 0, || Value::known(F::from(5)))
let b = region.assign_advice(|| "b", advice[1], 0, || Value::known(F::from(3)))
// ...
}
)?;
```

## PLONK vs halo2 비교

| 특징 | PLONK | halo2 |
|---|---|---|
| **기본 구조** | 3개 컬럼 (a, b, c) | 유연한 다중 컬럼 |
| **Gate 시스템** | 고정된 산술 Gate | 사용자 정의 Gate |
| **Lookup** | 기본적인 테이블 검색 | 복잡한 표현식 지원 |
| **재귀적 증명** | 제한적 지원 | 네이티브 지원 |

| **메모리 관리** | 단순한 행렬 | Region 기반 관리 |

| **최적화** | 기본적인 최적화 | 고급 최적화 기법 |

# PLONK 수학적 배경

## 1. 유한체 연산

모든 연산은 유한체 F_p에서 수행됩니다:

```
p = 2^254 + 4556048216821925013120110470117607854908139670496625 9
```

## 2. 타원곡선 쌍

증명 검증을 위해 타원곡선 쌍(pairing)을 사용:

```
e: G₁ × G₂ → G_T
```

## 3. 다항식 약속 (KZG)

Kate-Zaverucha-Goldberg 다항식 약속 스킴:

```
Commit(f(X)) = [f(τ)]₁ = f(τ) · G₁
```

여기서 τ는 신뢰할 수 있는 설정에서 생성된 비밀값입니다.

# halo2에서의 PLONK 프로토콜 실행

## 1. Setup 단계

```rust
// 공통 참조 문자열 생성
let params = ParamsKZG::<Bn256>::setup(k, OsRng);
let vk = keygen_vk(&params, &empty_circuit)?;
let pk = keygen_pk(&params, vk, &empty_circuit)?;
```

### 2. Prove 단계

```rust
// 증명 생성
let mut transcript = Blake2bWrite::<_, _, Challenge255<_>>::init(vec![]);
create_proof(&params, &pk, &[circuit], &[&[]], OsRng, &mut transcript)?;
let proof = transcript.finalize();
```

### 3. Verify 단계

```rust
// 증명 검증
let mut transcript = Blake2bRead::<_, _, Challenge255<_>>::init(&proof[..]);
let strategy = SingleVerifier::new(&params);
verify_proof(&params, pk.get_vk(), strategy, &[&[]], &mut transcript)
```

## PLONK의 보안성

### 1. 암호학적 가정

- **이산로그 문제**: 타원곡선에서의 이산로그 문제의 어려움

- **q-SDH 가정**: q-Strong Diffie-Hellman 가정
- **Knowledge of Exponent**: 지수 지식 가정

### 2. 보안 속성

- **완전성**: 올바른 증명은 항상 검증됨
- **건전성**: 잘못된 증명은 검증되지 않음
- **영지식성**: 증명에서 비밀 정보가 누출되지 않음

## 실제 구현에서의 고려사항

### 1. 성능 최적화

```
// FFT 최적화
let domain = EvaluationDomain::new(1, k);
let mut a_poly = domain.coeff_from_vec(a_values);
domain.fft(&mut a_poly);
```

### 2. 메모리 효율성

```
// 스트리밍 처리
let mut commitment = G1Affine::identity();
for chunk in polynomial.chunks(CHUNK_SIZE) {
let partial_commitment = compute_commitment(chunk);
commitment = commitment + partial_commitment;
```

```
}
```

### 3. 병렬 처리

```
// 병렬 다항식 연산
use rayon::prelude::*;
let  results: Vec<_> = polynomials.par_iter()
.map(|poly| compute_commitment(poly))
.collect();
```

## 다음 단계

PLONK와 halo2의 관계를 이해했다면:

- 다른 ZK 시스템과의 비교: halo2의 독특한 특징 파악
- 기본 개념: 구체적인 구현 개념 학습
- Circuit 설계: 실제 회로 구현 방법

---

> **참고**: PLONK는 halo2의 이론적 기반이며, halo2는 PLONK를 실용적으로 구현하고 확장한 시스템입니다. 두 시스템의 관계를 이해하면 halo2의 설계 철학을 더 잘 파악할 수 있습니다.